RBAC Permissions Matrix

RBAC Permissions Matrix

Branch: `feature/rbac-core` **Version**: 2.0 **Date**: 2026-07-11 **Scope**: Discord role → capability → UI tab → API route mapping

---

Table of Contents

Tier Definitions

Tier	Intended For	Capability Count	Summary
public	Unauthenticated visitors, external users	1	Read server status only
observer	Monitoring-only roles, DevOps watchers	3	Infrastructure health read: status, readiness, services
moderator	Community moderators, game helpers	11	Read-focused player/map/guild management, no writes
admin	Trusted administrators	16	Player/guild writes, broadcast, database schema access
owner	Server owner	21	Full access including destructive

Tier	Intended For	Capability Count	Summary
			ops, server control, RBAC admin

Tier Resolution

```
local password login  →  owner  (all capabilities, unconditionally)
Discord OAuth2 login  →  resolved by role IDs:
  1. DISCORD_OWNER_USER_IDS   →  owner
  2. guild member API fetch   →  role IDs matched against DISCORD_*_ROLE_IDS env vars
  3. CAPABILITY_BY_TIER[tier] →  static fallback
  DB override               →  dune.rbac_role_capabilities table overrides all
unknown source           →  public (empty set)
```

Capability Catalog

21 capabilities across 8 domains, exported as `DISCORD_CAPABILITIES` from `console/api/src/integrations/discord/policy.js`.

Status & Health

#	Key	String Value	Description
1	`STATUS_READ`	`status:read`	View server online/offline status
2	`READINESS_READ`	`readiness:read`	View server readiness checks
3	`SERVICES_READ`	`services:read`	View running services list

Population & Maps

#	Key	String Value	Description
4	`POPULATION_READ`	`population:read`	View player count and population data
5	`MAPS_READ`	`maps:read`	View map configurations and live map

Logs & Diagnostics

#	Key	String Value	Description
6	`LOGS_READ`	`logs:read`	View server logs

#	Key	String Value	Description
7	DIAGNOSTICS_READ	diagnostics:read	View diagnostic data and addon access

Database & Backups

#	Key	String Value	Description
8	BACKUPS_READ	backups:read	View backup list
9	BACKUPS_MANAGE	backups:manage	Create, restore, delete backups
10	DATABASE_READ	database:read	View database schema and table lists
11	DATABASE_QUERY	database:query	Execute arbitrary SQL queries

Player Management

#	Key	String Value	Description
12	PLAYERS_READ	players:read	View player profiles, stats, inventories
13	PLAYERS_WRITE	players:write	Give items, XP, skills, currency; modify inventory
14	PLAYERS_DELETE	players:delete	Delete inventory items, wipe inventories

Inventory & Storage

#	Key	String Value	Description
15	INVENTORY_READ	inventory:read	View player inventories
16	STORAGE_READ	storage:read	View shared/guild storage

Guild & Social

#	Key	String Value	Description
17	GUILD_READ	guild:read	View guild information
18	GUILD_WRITE	guild:write	Manage guild settings and Landsraad

Server Control

#	Key	String Value	Description
19	SERVER_CONTROL	server:control	Start, stop, restart server services
20	BROADCAST_SEND	broadcast:send	Send server-wide broadcast messages

Administration

#	Key	String Value	Description
21	AUTH_MANAGE	auth:manage	Manage RBAC roles, capabilities, and audit log

Write-Gated Capabilities

7 capabilities are gated behind DUNE_DISCORD_WRITES_ENABLED=true for the Discord bot: broadcast:send, backups:manage, database:query, players:write, players:delete, guild:write, server:control

The remaining 14 are read-only and always available.

Tier-to-Capability Matrix

#	Capability	public	observer	moderator	admin	owner
1	status:read	✓	✓	✓	✓	✓
2	readiness:read		✓	✓	✓	✓
3	services:read		✓	✓	✓	✓
4	population:read			✓	✓	✓
5	maps:read			✓	✓	✓
6	logs:read				✓	✓
7	diagnostics:read				✓	✓
8	backups:read			✓	✓	✓
9	backups:manage					✓
10	database:read				✓	✓
11	database:query					✓
12	players:read			✓	✓	✓
13	players:write				✓	✓
14	players:delete					✓
15	inventory:read			✓	✓	✓
16	storage:read			✓	✓	✓
17	guild:read			✓	✓	✓
18	guild:write				✓	✓
19	server:control					✓
20	broadcast:send				✓	✓
21	auth:manage					✓

UI Tab Access Matrix

17 tabs across 3 nav groups. Tabs are hidden from the sidebar when the user lacks the required capability.

Server Operations

Tab	Required Capability	public	observer	moderator	admin	owner
Home	`status:read`	✓	✓	✓	✓	✓
Services	`services:read`		✓	✓	✓	✓
Logs	`logs:read`				✓	✓
Backups (list)	`backups:read`			✓	✓	✓
Backups (manage)	`backups:manage`					✓
Database (schema)	`database:read`				✓	✓
Database (query)	`database:query`					✓
Updates	`server:control`					✓
Settings	`server:control`					✓
Server Control	`server:control`					✓

Arrakis Management

Tab	Required Capability	public	observer	moderator	admin	owner
Players (read)	`players:read`			✓	✓	✓
Players (write)	`players:write`				✓	✓
Players (delete)	`players:delete`					✓
Admin Tools	`players:write`				✓	✓
Care Package	`players:write`				✓	✓
Live Map	`maps:read`			✓	✓	✓
Maps	`maps:read`			✓	✓	✓
Guilds (read)	`guild:read`			✓	✓	✓
Guilds (write)	`guild:write`				✓	✓
Landsraad	`guild:write`				✓	✓
Storage	`storage:read`			✓	✓	✓

Community

Tab	Required Capability	public	observer	moderator	admin	owner
Addons	`diagnostics:read`				✓	✓

RBAC Admin

Tab	Required Capability	public	observer	moderator	admin	owner
RBAC Roles	`auth:manage`					✓
RBAC Audit	`auth:manage`					✓

Design Notes

- **Destructive operations** (`players:delete`, `backups:manage`, `database:query`, `server:control`) are owner-only
- **Player modifiers** (`players:write`, `guild:write`) are admin+. A moderator can view player data but cannot modify it
- **Content tools** (Admin Tools, Care Package) are admin+ since they can directly impact player progression
- **Database queries** are owner-only — SQL injection risk demands maximum restriction
- **Backups** have split read/write: listing is moderator+ (awareness of backup state), management is owner-only
- **Server control** (start/stop/restart/config) is owner-only — infrastructure stability
- **Broadcast** is admin+ — trusted messaging to all online players
- **Landsraad** follows `guild:write` since it requires guild management privileges
- Tabs are **hidden** (not just disabled) when capability is absent, preventing confusion over grayed-out UI
- `public` tier sees ONLY the Home tab and the login page
- `observer` tier is intentionally narrow: health monitoring only, no player data access

---

Route-to-Capability Registration

Registered in `server.js` via `registerRoute(method, pattern, capability)`. Unregistered routes pass through (migration in progress). Pattern parameters (`:param`) are converted to regex for matching.

Method	Route Pattern	Capability
POST	`/api/players/:id/give-item`	`players:write`
POST	`/api/players/:id/give-items`	`players:write`
POST		`players:write`

Method	Route Pattern	Capability
	/api/players/:id/give-item-id	
POST	/api/players/:id/add-currency	players:write
POST	/api/players/:id/add-faction-reputation	players:write
POST	/api/players/:id/add-intel	players:write
POST	/api/players/:id/augment-item	players:write
POST	/api/players/:id/clear-augments	players:write
POST	/api/players/:id/inventory/:itemId	players:write
DELETE	/api/players/:id/inventory/:itemId	players:delete
POST	/api/players/:id/clean-inventory	players:delete
POST	/api/players/:id/specializations/add-xp	players:write
POST	/api/players/:id/specializations/grant-max	players:write
POST	/api/players/:id/specializations/reset	players:write
POST	/api/server/restart	server:control
POST	/api/server/start	server:control
POST	/api/server/stop	server:control
POST	/api/server/update	server:control
POST	/api/backups/create	backups:manage
POST	/api/backups/restore	backups:manage
DELETE	/api/backups/:id	backups:manage
POST	/api/database/query	database:query
POST	/api/settings	server:control
PUT	/api/rbac/roles/:id	auth:manage

Denial Behavior

When a capability check fails: - **HTTP 403** returned: `{ ok: false, code: "not_authorized", error: "Not authorized. Required capability: {capability}." }` - **Audit logged**: `auth.capability.denied` event with actor, capability, route, and timestamp to `dune.rbac_audit_log`

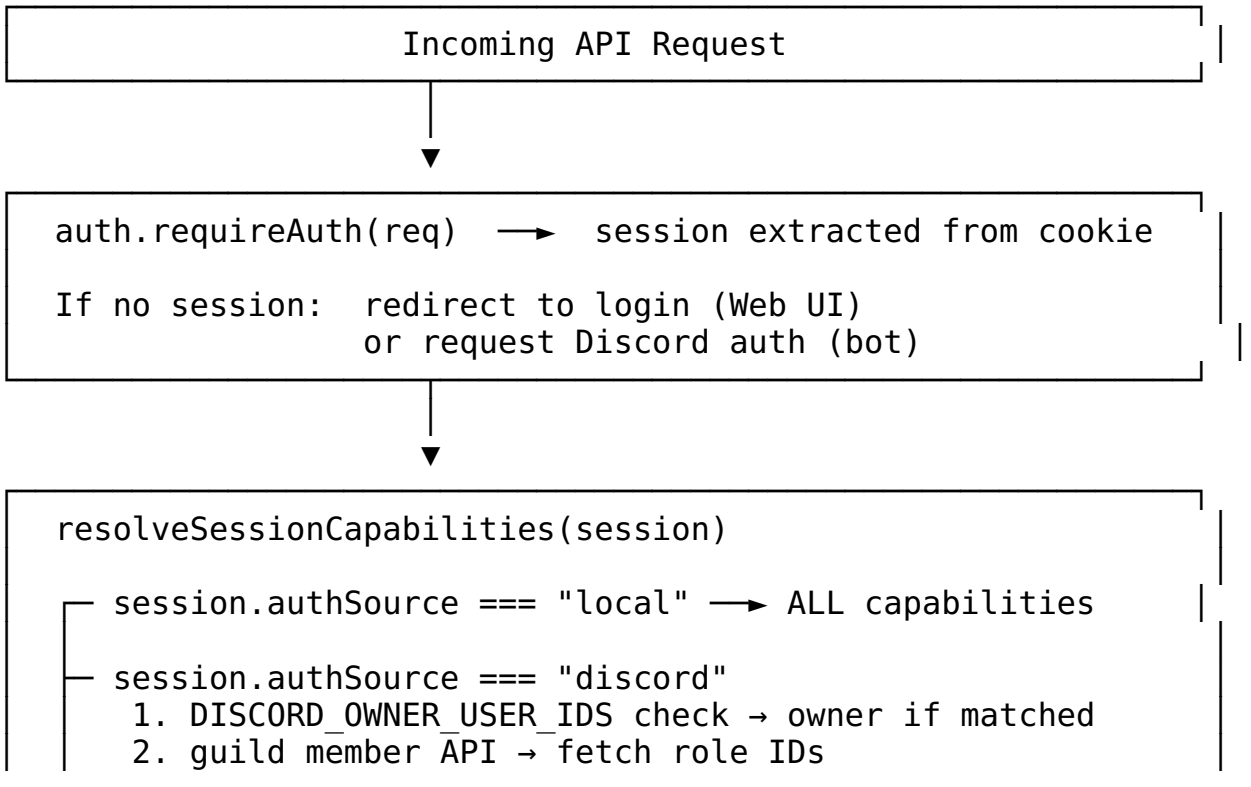
Discord Adapter Route Mapping

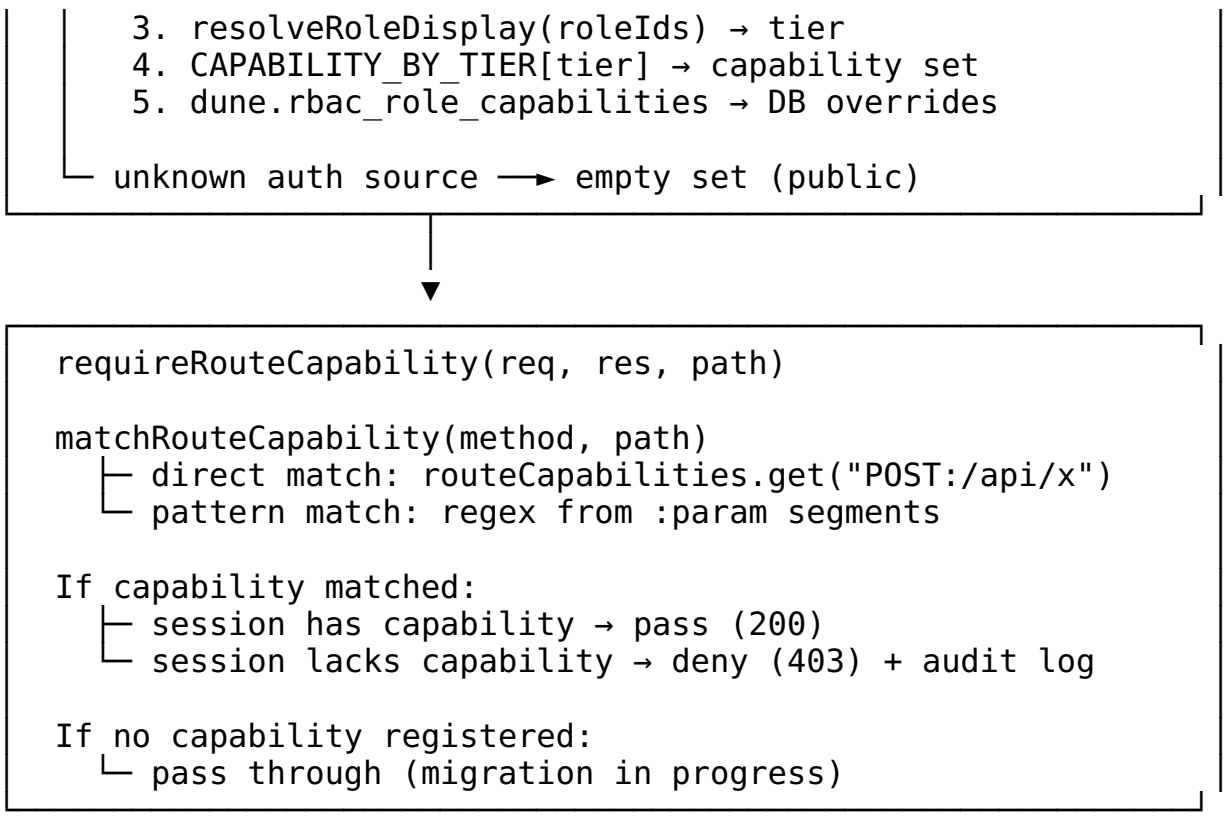
Routes called by the Discord bot (not the Web UI). Capabilities checked via `requireDiscordCapability()`.

Method	Route	Capability
GET	`/api/integrations/discord/health`	None (always allowed)
POST	`/api/integrations/discord/status`	`status:read` or `logs:read`
POST	`/api/integrations/discord/readiness`	`readiness:read`
POST	`/api/integrations/discord/services`	`services:read`
POST	`/api/integrations/discord/population`	`population:read`
POST	`/api/integrations/discord/servers`	`services:read`
POST	`/api/integrations/discord/ports`	`services:read`
POST	`/api/integrations/discord/db`	`services:read`
POST	`/api/integrations/discord/ops/*`	`ops:read`
POST	`/api/integrations/discord/players/link`	`inventory:read`
POST	`/api/integrations/discord/players/unlink`	`inventory:read`
POST	`/api/integrations/discord/players/me`	`inventory:read`
POST		`inventory:read`

Method	Route	Capability
	/api/integrations/discord/players/inventory	
POST	/api/integrations/discord/players/storage	storage:read
POST	/api/integrations/discord/players/find	inventory:read
POST	/api/integrations/discord/players/inventory-search	inventory:read
POST	/api/integrations/discord/guilds/storage	guild:read
POST	/api/integrations/discord/guilds/find	guild:read

Session Resolution Flow





Environment Variables

Required for Role-Based Access

Variable	Description
`DISCORD_GUILD_ID`	Discord guild ID for role lookup (`guilds.members.read` scope required)
`DISCORD_OWNER_ROLE_IDS`	Comma-separated Discord role IDs mapped to owner tier
`DISCORD_ADMIN_ROLE_IDS`	Comma-separated Discord role IDs mapped to admin tier
`DISCORD_MODERATOR_ROLE_IDS`	Comma-separated Discord role IDs mapped to moderator tier
`DISCORD_OBSERVER_ROLE_IDS`	Comma-separated Discord role IDs mapped to observer tier

Optional

Variable	Description
`DISCORD_OAUTH_CLIENT_ID`	Discord application client ID (disable to use local password only)
`DISCORD_OAUTH_CLIENT_SECRET`	Discord application client secret
`DISCORD_OAUTH_REDIRECT_URI`	OAuth2 callback URL
`DISCORD_OWNER_USER_IDS`	

Variable	Description
	Fallback: comma-separated Discord user IDs directly mapped to owner
`DUNE_DISCORD_WRITES_ENABLED`	Gate write capabilities for the Discord bot (`true`/`false`)

Secret File Locations

Env vars take priority. If not set, values are loaded from `runtime/secrets/`:

Env Var	Secret File
`DISCORD_OAUTH_CLIENT_ID`	`runtime/secrets/discord-oauth-client-id.txt`
`DISCORD_OAUTH_CLIENT_SECRET`	`runtime/secrets/discord-oauth-client-secret.txt`
`DISCORD_OAUTH_REDIRECT_URI`	`runtime/secrets/discord-oauth-redirect-uri.txt`
`DISCORD_GUILD_ID`	`runtime/secrets/discord-guild-id.txt`

Database Tables

`dune.rbac_role_capabilities`

Runtime-overridable capability assignments. Entries here override `CAPABILITY_BY_TIER` static mappings.

```
CREATE TABLE IF NOT EXISTS dune.rbac_role_capabilities (
    role_id    TEXT NOT NULL,
    capability TEXT NOT NULL,
    granted_by TEXT,
    granted_at TIMESTAMPTZ DEFAULT now(),
    PRIMARY KEY (role_id, capability)
);
```

Managed via: - `GET /api/rbac/roles` — list all role-capability mappings (requires `auth:manage`) - `PUT /api/rbac/roles/:id` — set capabilities for a role (requires `auth:manage`)

`dune.rbac_audit_log`

Immutable audit trail for all RBAC-significant events.

```
CREATE TABLE IF NOT EXISTS dune.rbac_audit_log (
    id          BIGSERIAL PRIMARY KEY,
    actor_id    TEXT NOT NULL,
    actor_name  TEXT,
    action      TEXT NOT NULL,
    target_type TEXT,
```

```
    target_id   TEXT,
    route       TEXT,
    result      TEXT DEFAULT 'success',
    detail      JSONB DEFAULT '{}',
    timestamp   TIMESTAMPTZ DEFAULT now()
);
```

Managed via: - `GET /api/rbac/audit` — view last 200 audit entries (requires `auth:manage`)

dune.discord_user_profiles

Discord user identity cache — populated on each OAuth2 login.

```
CREATE TABLE IF NOT EXISTS dune.discord_user_profiles (
    discord_user_id TEXT PRIMARY KEY,
    username        TEXT NOT NULL,
    avatar_hash     TEXT,
    auth_source     TEXT DEFAULT 'discord',
    last_login_at   TIMESTAMPTZ DEFAULT now()
);
```

dune.discord_player_links

Explicit Discord-to-game-character linking for bot commands.

```
CREATE TABLE IF NOT EXISTS dune.discord_player_links (
    discord_user_id      TEXT PRIMARY KEY,
    player_controller_id TEXT NOT NULL,
    faction              TEXT DEFAULT 'fremen',
    linked_at            TIMESTAMPTZ DEFAULT now()
);
```